Máster en Seguridad TIC (XIII edición)

Reporte de Laboratorio

P01: Criptografía básica

Autor: Juan Antonio Robledo Menéndez
Tutor: Antonio Estepa Alonso
Fecha de realización: 20 de octubre de 2025
Fecha de entrega: 4 de noviembre de 2025
Módulo: Introducción a la Seguridad e Identificación Digital

Universidad de Sevilla
Curso 2025-2026

Índice

1 Ejercicio simple de cifrado simétrico **2**

2 Comparativa prestaciones de cifrado simétrico **6**

3 Ejercicio simple de cifrado asimétrico **13**
3.1 Generación de claves pública y privada 13
3.2 Cifrado y descifrado con un compañero 15

4 Ejercicio simple de firma electrónica **16**
4.1 Generación de claves públicas y privadas en gpg 17

5 Ejercicio final **20**

1. Ejercicio simple de cifrado simétrico

En primer lugar se generará un fichero que contendrá un mensaje de texto en claro y al que someteremos, a lo largo de la práctica, a diferentes algoritmos de cifrado. Se denominará `M.txt`.

```
(dit@lt205)-[~/cifrado]
$ echo 'Este es un mensaje en claro' > M.txt
```

Podemos visualizar el contenido byte a byte empleando el comando `hexdump`, donde el primer bloque de caracteres representa el offset en hexadecimal (número de bytes desde el inicio del fichero) y el segundo, la conversión de los bytes en ASCII (debido al parámetro `-c`; en ausencia de este último el contenido se mostraría en formato hexadecimal).

```
(dit@lt205)-[~/cifrado]
$ hexdump -c M.txt
0000000   E  s  t  e     e  s     u  n     m  e  n  s  a
0000010   j  e     e  n     c  l  a  r  o \n
000001c
```

Para este apartado emplearemos el programa `GNU Privacy Guard`, el cual es una implementación libre y gratuita del estándar `OpenPGP` (RFC 4880). Sirve principalmente para cifrar/descifrar y firmar digitalmente archivos, además de gestionar claves públicas y privadas. Funciona bajo un modelo de confianza descentralizado denominado 'web de confianza'[1].

```
(dit@lt205)-[~/cifrado]
$ gpg --version
gpg (GnuPG) 2.4.7
libgcrypt 1.11.0
Copyright (C) 2024 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /home/dit/.gnupg
Algoritmos disponibles:
Clave publica: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cifrado: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
         CAMELLIA128, CAMELLIA192, CAMELLIA256
Resumen: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compresion: Sin comprimir, ZIP, ZLIB, BZIP2
```

Como se puede apreciar en el comando anterior, la versión del programa empleado es la 2.4.7, la cual soporta diferentes algoritmos, algunos de ellos vistos en teoría, como por ejemplo: RSA, EC, 3DES, AES, SHA... Algunos serán empleados exclusivamente en el

[1] No existen las PKI, se soporta a base de crear una red de firmas. Inicia en el establecimiento de una relación de confianza con un individuo, si posteriormente se quiere confiar en un tercero, aquel en el que se confia debe pasar la clave pública del tercero firmada digitalmente con su clave privada.

cifrado, otros en la generación de claves públicas o en la compresión.

El siguiente comando permite cifrar el fichero `M.txt` mediante el algoritmo 3DES, empleando una `passphrase`, la cual no se trata de la clave privada; GPG la utiliza sólo para crear una clave adecuada para cada algoritmo (tal y como se cita en el enunciado de la práctica). La bandera `-allow-old-ciopher-algos` es necesaria añadirla, puesto que en versiones actuales GPG considera 3DES un algoritmo inseguro.

```
(dit@lt205)-[~/cifrado]
$ gpg --allow-old-cipher-algos -c --cipher-algo  3DES M.txt
```

Si se detalla el nombre del archivo de salida (con la bandera `-o`), generará uno con el mismo nombre con la extensión `.gpg`. Otra cuestión a destacar es que el tamaño del archivo generado es distinto al original (28 y 94 bytes), esto se debe a dos razones explicadas en el enunciado de la práctica, y es que por una parte GPG añade una cabecera a la información codificada; además comprime el contenido del fichero original (`M.txt`) antes de cifrar si no se indica lo contrario.

```
(dit@lt205)-[~/cifrado]
$ ls -l
total 8
-rw-rw-r-- 1 dit dit 28 oct 24 17:16 M.txt
-rw-rw-r-- 1 dit dit 94 oct 24 17:35 M.txt.gpg
```

Se puede ver el formato del archivo GPG y sus cabeceras mediante la ejecución del siguiente comando:

```
(kali@kali)-[~/ciberseguridad]
$ gpg --list-packets M.txt.gpg
gpg: 3DES.CFB encrypted data
gpg: encrypted with 1 passphrase
# off=0 ctb=8c tag=3 hlen=2 plen=13
:symkey enc packet: version 4, cipher 2, aead 0, s2k 3, hash 10
        salt DB4DDBC933B61A20, count 65011712 (255)
# off=15 ctb=d2 tag=18 hlen=2 plen=78 new-ctb
:encrypted data packet:
        length: 78
        mdc_method: 2
# off=28 ctb=a3 tag=8 hlen=1 plen=0 indeterminate
:compressed packet: algo=1
# off=30 ctb=ac tag=11 hlen=2 plen=39
:literal data packet:
        mode b (62), created 1761667932, name="M.txt",
        raw data: 28 bytes
```

La primera línea de la salida del comando nos indica que se ha empleado para encriptar el mensaje el algoritmo de cifrado simétrico 3DES-CFB. Por otro lado, la segunda línea muestra que GPG no usó claves públicas, sino una `passphrase` para derivar una clave simétrica y añadir entropía. Cabe destacar la presencia del parámetro `salt`, se trata de

un valor aleatorio empleado junto con la contraseña (`passphrase`) para derivar la clave de sesión. Esto garantiza que aunque dos individuos tengan la misma `passphrase` (PF), las claves de sesión serán completamente distintas. Si se supone que dos usan la misma PF, sin salt, derivarán la misma clave de sesión, y si cifran un mismo archivo idéntico, obtendrán el mismo resultado, lo que permitiría a los atacantes detectar patrones. En la sección de `encrypted data packet` podemos ver el tamaño total del mensaje encriptado y el identificador del algoritmo de compresión utilizado, en este caso 2 equivale a SHA1.

A continuación, se va a cifrar el contenido del fichero `M.txt` mediante el algoritmo de cifrado AES128-CFB (sin realizar compresión debido a la presencia de la bandera `-z 0`) y el resultado se va a almacenar en el fichero `C.AES`.

```
(dit@lt205)-[~/cifrado]
$ gpg -o C.AES -c --cipher-algo AES -z 0 M.txt
```

Al listar el contenido de la carpeta `cifrado` se puede comprobar ques se ha generado el fichero previamente mencionado. Cabe destacar que el `C.AES` ocupa 5 bytes más que el mensaje cifrado anteriormente mediante el algoritmo 3DES, principalmente porque el primero emplea bloques de 128 bits y el otro de 64 bits, lo cual afecta al tamaño del relleno (padding) y al vector de inicialización (IV); además, el mensaje en este caso no ha sido comprimido (la corta extensión del mensaje hace que la compresión sea mínima).

```
(dit@lt205)-[~/cifrado]
$ ls -l
total 12
-rw-rw-r-- 1 dit dit 99 oct 24 17:38 C.AES
-rw-rw-r-- 1 dit dit 28 oct 24 17:16 M.txt
-rw-rw-r-- 1 dit dit 94 oct 24 17:35 M.txt.gpg
```

Si se quisiera descifrar el contenido de un mensaje encriptado por otro individuo (como es el caso del `C_Marga.AES`), se emplearía el comando situado abajo. Es importante destacar que para desencriptar no es necesario especificar el algoritmo con el se cifró el mensaje original (gracias a las cabeceras del `.gpg`), tan solo sería necesario la `passphrase` impuesta por el individuo.

```
(dit@lt205)-[~/cifrado]
$ gpg -o C_Marga.txt -d C_Marga.AES
gpg: AES.CFB encrypted data
gpg: cifrado con 1 frase contrasena
```

¿Qué necesitas saber para descifrar C.AES?¿y tu compañero/a?¿cómo podríais compartir esa información en un entorno no fiable?

Como se ha comentado con anterioridad, se necesitaría exclusivamente la `passphrase` que introdujo mi compañera, el resto de la información necesaria para desencriptar se encuentra en la cabecera del archivo `.gpg`.
Mi compañera, por otro lado, necesitaría mi `passphrase` para descifrar mi mensaje.
Lo más lógico sería emplear un cifrado asimétrico, donde ambos individuos podrían pasar sus claves públicas firmadas digitalmente; para posteriormente encriptar sus

respectivas `passphrases` con la clave pública del otro; sin embargo, eso no garantiza completa autenticidad, puesto que se necesitarían certificados y una PKI. Por ello, lo más correcto sería utilizar el modelo de web de confianza, donde ambos individuos deban establecer una relación de confianza con un tercero, que les pasá la clave pública del otro firmada digitalmente por él (posteriormente se intercambiarían las `passphrases`).

Escriba un nuevo mensaje. Cífrelo con salida en ASCII. Copie y pegue el mensaje cifrado en un email enviado a su compañero. Reciba el email de su compañero y descífrelo.

A efectos prácticos, los comandos empleados para la realización del ejercicio son idénticos a los que ya se han mostrado previamente, simplemente se debe tener en cuenta que se debe pasar el mensaje cifrado mediante un email.
En primer lugar procedemos a crear el mensaje. Se almacenará en el `prueba.txt`.

```
(dit@lt205)-[~/cifrado]
$ echo 'Ejericio simple de cifrado simetrico' > prueba.txt
```

Luego ciframos el fichero mediante el algoritmo AES128-CFB. Es imprescindible el uso de la bandera `-armor` porque como se ha comentado con anterioridad, el mensaje cifrado debe ser enviado por email, por lo que sería imposible si el contenido del fichero estuviera formado por caracteres no ASCII, podrían corromperse y verse como símbolos raros (romper el mensaje). El proceso de codificación en ASCII consiste en la codificación en `Radix-64` del dato en binario (se cogen 6 bits y se mapea a un caracter ASCII, en un rango de valores en decimal de 0 a 63).

```
(dit@lt205)-[~/cifrado]
$ gpg --armor -o prueba.asc -c --cipher-algo AES -z 0 prueba.txt
```

Se puede comprobar que se ha codificado correctamente mediante `Radix-64`, si al imprimir por la terminal, mediante el comando `cat`, el contenido del fichero cifrado se muestra sin caracteres no ASCII.

```
(dit@lt205)-[~/cifrado]
$ cat prueba.asc
-----BEGIN PGP MESSAGE-----

jA0EBwMKf5rikaVzOAD/0mEBxDSbmLyE0/ZItamouyuVNdzEnECMdyr2ZdFTW2e8
hNDLFu0bJfghyk/xZJILy4m2Yobu+5xYoNcUE7FljcpJNjuncheBpng4dVAB4kQN
izczUVqqG4S16hipEnUa57Sm
=Km7x
-----END PGP MESSAGE-----
```

Por último, se ha descifrado el mensaje encriptado del compañero (pasado por email) mediante su `passphrase` y luego se ha mostrado su contenido.

```
(dit@lt205)-[~/cifrado]
$ gpg -o prueba_compi_descifrada.txt -d prueba_compi.asc
```

```
gpg: AES.CFB encrypted data
gpg: encrypted with 1 passphrase
```

```
(dit@lt205)-[~/cifrado]
$ cat prueba_compi_descifrada.txt
Ejericio simple de cifrado simetrico del compi
```

2. Comparativa prestaciones de cifrado simétrico

Esta parte del reporte se centrará en `OpenSSL`, la cual es una biblioteca de software y una herramienta de línea de comandos que implementa funciones criptográficas para cifrar, descifrar, firmar, verificar, generar claves y certificados digitales entre otras tareas de seguridad.

Si ejecutamos el siguiente comando se puede observar el conjunto de algoritmos que soporta. La primera columna representa el nombre completo de la `cipher suite`, la segunda la versión mínima de TLS/SSL donde está disponible, la tercera el algoritmo de intercambio de claves (cómo se establece la clave compartida), la siguiente el algoritmo usado para autenticar el servidor o cliente, la penúltima el algoritmo de cifrado simétrico usado para proteger los datos y finalmente el método para verificar la integridad de los mensajes.

```
(dit@lt205)-[~/cifrado]
$ openssl ciphers -v
TLS_AES_256_GCM_SHA384 TLSv1.3 Kx=any Au=any Enc=AESGCM(256) Mac=AEAD
TLS_AES_128_GCM_SHA256 TLSv1.3 Kx=any Au=any Enc=AESGCM(128) Mac=AEAD
```

El siguiente comando permite (al igual que en GPG) cifrar el fichero `M.txt` mediante el algoritmo AES256-CBC y almacenar la salida en el fichero `C.enc`. La particularidad reside en la bandera `-salt` la cual hace que `OpenSSL` no realice una generación determinista (o sea, que se genere una clave y vector de iniciación diferente cada vez se escriba la misma frase secreta).

```
openssl enc -aes-256-cbc -salt -in M.txt -out C.enc
```

Se puede comprobar el valor usado para la clave del algoritmo, así como el vector de iniciación usando la opción `-p` al final del comando anterior.

```
(dit@lt205)-[~/cifrado]
$ openssl enc -aes-256-cbc -salt -in M.txt -out C.enc -p
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
salt=B0E7198DA7F22C53
key=E5815019B253C1B8B11AF689C0B3F2F5D241F40CAE3D5C67945715E53D7DEE7E
iv =75D0695301D09125FA356AA3CBC381D2
```

Para desencriptar, en cambio, se emplea el siguiente comando.

```
(dit@lt205)-[~/cifrado]
$ openssl aes-256-cbc -d -in C.enc -out M_bis.txt
```

```
(dit@lt205)-[~/cifrado]
$ cat M_bis.txt
Este es un mensaje en claro
```

Realice una comparación de la velocidad de cifrado de AES128 / AES256 / DES y triple DES de dos claves (des-ede3) en el mismo modo de operación (p.ej. CBC).

Para poder comparar la velocidad de cifrado de los distintos algoritmos se empleará el comando `speed`. Su función reside en cifrar una cantidad de bits variable y determinada durante 3 segundos.
En primer lugar se procederá con el algoritmo DES-CBC:

```
(dit@lt205)-[~/cifrado]
$ openssl speed des-cbc
Doing des-cbc ops for 3s on 16 size blocks:
15001701 des-cbc ops in 3.00s
Doing des-cbc ops for 3s on 64 size blocks:
4125940 des-cbc ops in 3.00s
Doing des-cbc ops for 3s on 256 size blocks:
1054159 des-cbc ops in 3.00s
Doing des-cbc ops for 3s on 1024 size blocks:
264875 des-cbc ops in 3.00s
Doing des-cbc ops for 3s on 8192 size blocks:
33101 des-cbc ops in 3.00s
Doing des-cbc ops for 3s on 16384 size blocks:
16496 des-cbc ops in 3.00s
version: 3.5.1
built on: Sat Jul 12 16:49:06 2025 UTC
options: bn(64,64)
compiler: gcc -fPIC -pthread -m64 -Wa,--noexecstack -Wall
-fzero-call-used-regs=used-gpr -Wa,--noexecstack -g -O2
-Werror=implicit-function-declaration
-ffile-prefix-map=/build/reproducible-path/openssl-3.5.1=.
-fstack-protector-strong -fstack-clash-protection -Wformat
-Werror=format-security -fcf-protection -DOPENSSL_USE_NODELETE
-DL_ENDIAN -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DZLIB
-DZSTD -DNDEBUG -Wdate-time -D_FORTIFY_SOURCE=2
CPUINFO: OPENSSL_ia32cap=0x7ffaf3bfffebffff:0x40000000029c67af
:0x00000000bc000600:0x0000000000000000:0x0000000000000000
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes
8192 bytes  16384 bytes
```

```
des-cbc          80009.07k    88020.05k    89954.90k
90410.67k
90387.80k    90090.15k
```

El throughput se calcula como $T = \frac{N \cdot B}{t}$, donde N es el número de operaciones, B el tamaño del bloque (en bytes) y t el tiempo de ejecución (en segundos). Las primeras líneas de la salida del comando muestran los valores de las variables recién explicadas, por ejemplo, en la primera medida el número de operaciones sería 15001701, el número de bytes sería de 16 y el tiempo de 3 segundos, dando un caudal de 80009.07 kB (tal y como se muestra en la parte final de la salida del comando).

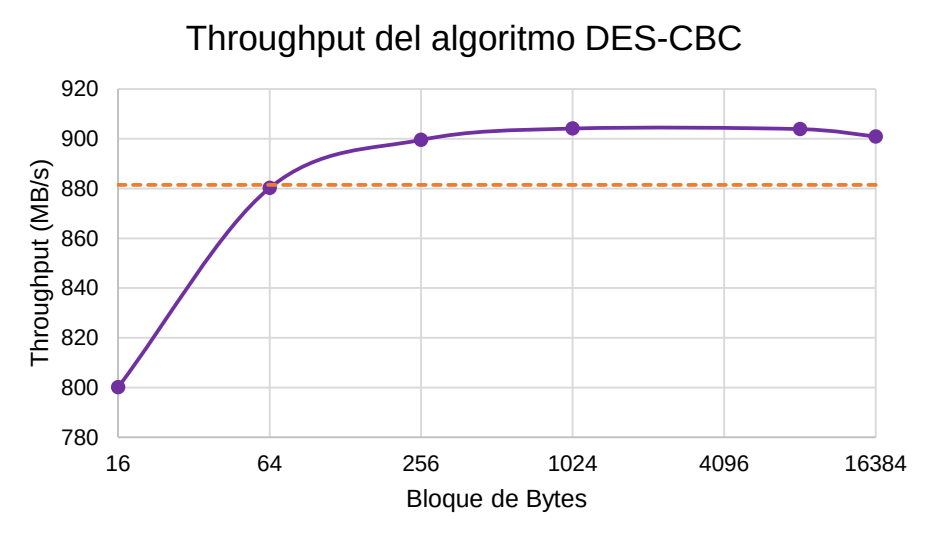

Figura 1: Throughput del algoritmo DES-CBC en función del tamaño de bloque (con línea de media).

A continuación se procederá con el algoritmo AES-128-CBC:

```
(dit@lt205)-[~/cifrado]
$ openssl speed aes-128-cbc
Doing aes-128-cbc ops for 3s on 16 size blocks:
201622661 aes-128-cbc ops in 2.99s
Doing aes-128-cbc ops for 3s on 64 size blocks:
71792848 aes-128-cbc ops in 3.00s
Doing aes-128-cbc ops for 3s on 256 size blocks:
18280998 aes-128-cbc ops in 3.00s
Doing aes-128-cbc ops for 3s on 1024 size blocks:
4608206 aes-128-cbc ops in 3.00s
Doing aes-128-cbc ops for 3s on 8192 size blocks:
580709 aes-128-cbc ops in 3.00s
Doing aes-128-cbc ops for 3s on 16384 size blocks:
290739 aes-128-cbc ops in 3.00s
version: 3.5.1
built on: Sat Jul 12 16:49:06 2025 UTC
options: bn(64,64)
compiler: gcc -fPIC -pthread -m64 -Wa,--noexecstack -Wall
```

```
-fzero-call-used-regs=used-gpr -Wa,--noexecstack -g -O2
-Werror=implicit-function-declaration
-ffile-prefix-map=/build/reproducible-path/openssl-3.5.1=.
-fstack-protector-strong -fstack-clash-protection -Wformat
-Werror=format-security -fcf-protection -DOPENSSL_USE_NODELETE
-DL_ENDIAN -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DZLIB
-DZSTD -DNDEBUG -Wdate-time -D_FORTIFY_SOURCE=2
CPUINFO: OPENSSL_ia32cap=0x7ffaf3bfffebffff:0x40000000029c67af
:0x00000000bc000600:0x0000000000000000:0x0000000000000000
The 'numbers' are in 1000s of bytes per second processed.
type              16 bytes     64 bytes    256 bytes   1024 bytes
8192 bytes  16384 bytes
aes-128-cbc    1078917.25k  1531580.76k  1559978.50k
1572934.31k
1585722.71k  1587822.59k
```

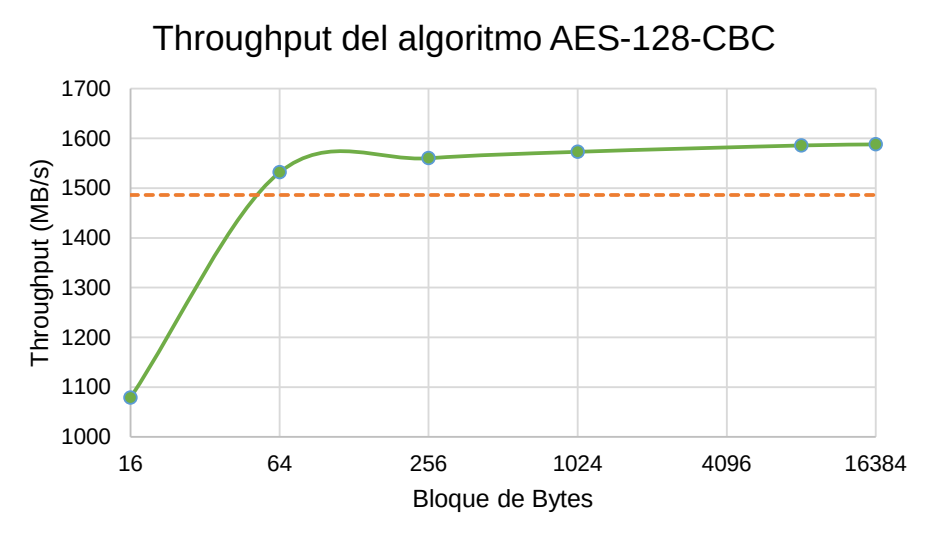

Figura 2: Throughput del algoritmo AES-128-CBC en función del tamaño de bloque (con línea de media).

Se procederá con el algoritmo AES-256-CBC:

```
(dit@lt205)-[~/cifrado]
$ openssl speed aes-256-cbc
Doing aes-256-cbc ops for 3s on 16 size blocks:
181175035 aes-256-cbc ops in 3.00s
Doing aes-256-cbc ops for 3s on 64 size blocks:
52102849 aes-256-cbc ops in 3.00s
Doing aes-256-cbc ops for 3s on 256 size blocks:
13008915 aes-256-cbc ops in 3.00s
Doing aes-256-cbc ops for 3s on 1024 size blocks:
3319309 aes-256-cbc ops in 3.00s
Doing aes-256-cbc ops for 3s on 8192 size blocks:
416832 aes-256-cbc ops in 3.00s
```

```
Doing aes-256-cbc ops for 3s on 16384 size blocks:
208932 aes-256-cbc ops in 3.00s
version: 3.5.1
built on: Sat Jul 12 16:49:06 2025 UTC
options: bn(64,64)
compiler: gcc -fPIC -pthread -m64 -Wa,--noexecstack -Wall
-fzero-call-used-regs=used-gpr -Wa,--noexecstack -g -O2
-Werror=implicit-function-declaration
-ffile-prefix-map=/build/reproducible-path/openssl-3.5.1=.
-fstack-protector-strong -fstack-clash-protection -Wformat
-Werror=format-security -fcf-protection -DOPENSSL_USE_NODELETE
-DL_ENDIAN -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DZLIB
-DZSTD -DNDEBUG -Wdate-time -D_FORTIFY_SOURCE=2
CPUINFO: OPENSSL_ia32cap=0x7ffaf3bfffebffff:0x40000000029c67af
:0x00000000bc000600:0x0000000000000000:0x0000000000000000
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes
8192 bytes  16384 bytes
aes-256-cbc     966266.85k  1111527.45k  1110094.08k
1132990.81k
1138229.25k  1141047.30k
```

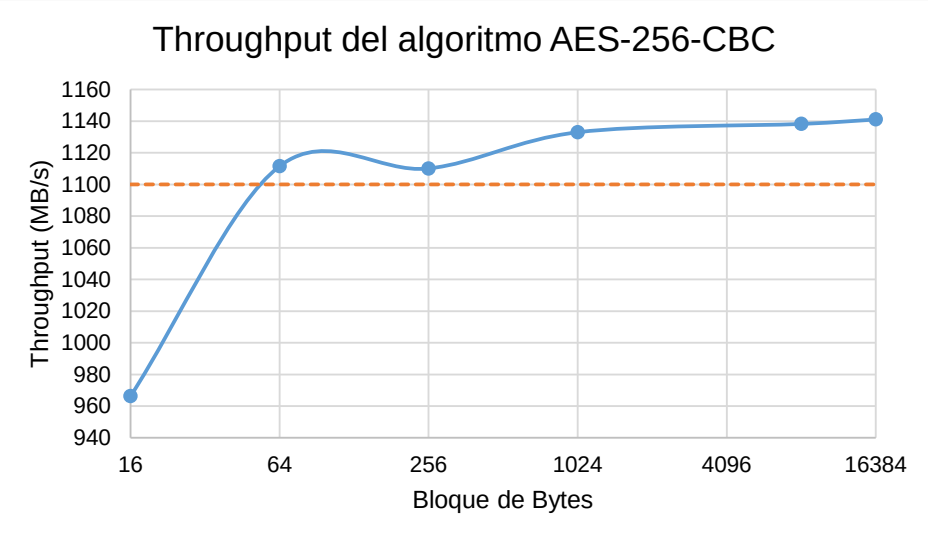

Figura 3: Throughput del algoritmo AES-256-CBC en función del tamaño de bloque (con línea de media).

Se finalizará con el algoritmo 3DES-CBC:

```
(kali@kali)-[~/ciberseguridad]
$ openssl speed -evp des-ede3-cbc
Doing DES-EDE3-CBC ops for 3s on 16 size blocks:
2794102 DES-EDE3-CBC ops in 3.03s
Doing DES-EDE3-CBC ops for 3s on 64 size blocks:
719807 DES-EDE3-CBC ops in 3.02s
```

```
Doing DES-EDE3-CBC ops for 3s on 256 size blocks:
179902 DES-EDE3-CBC ops in 3.02s
Doing DES-EDE3-CBC ops for 3s on 1024 size blocks:
45031 DES-EDE3-CBC ops in 3.02s
Doing DES-EDE3-CBC ops for 3s on 8192 size blocks:
5613 DES-EDE3-CBC ops in 3.02s
Doing DES-EDE3-CBC ops for 3s on 16384 size blocks:
2840 DES-EDE3-CBC ops in 3.02s
version: 3.5.2
built on: Sun Aug 10 09:30:37 2025 UTC
options: bn(64,64)
compiler: gcc -fPIC -pthread -m64 -Wa,--noexecstack -Wall
-fzero-call-used-regs=used-gpr -Wa,--noexecstack -g -O2
-Werror=implicit-function-declaration
-ffile-prefix-map=/build/reproducible-path/openssl-3.5.2=.
-fstack-protector-strong -fstack-clash-protection -Wformat
-Werror=format-security -fcf-protection -DOPENSSL_USE_NODELETE
-DL_ENDIAN -DOPENSSL_PIC -DOPENSSL_BUILDING_OPENSSL -DZLIB
-DZSTD -DNDEBUG -Wdate-time -D_FORTIFY_SOURCE=2
CPUINFO: OPENSSL_ia32cap=0x9ed83203078bffff:0x0000000000000000
:0x0000000000000000:0x0000000000000000:0x0000000000000000
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes
8192 bytes  16384 bytes
DES-EDE3-CBC     14754.33k    15254.19k    15249.97k
15268.79k
15225.73k    15407.47k
```

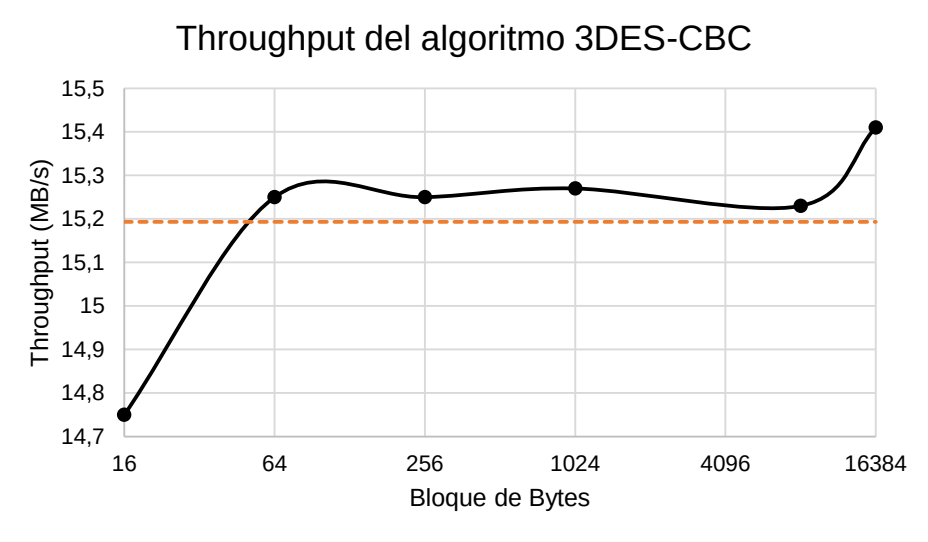

Figura 4: Throughput[2] del algoritmo 3DES-CBC en función del tamaño de bloque (con línea de media).

Si se observan las diferentes gráficas de los distintos algoritmos, se puede apreciar una clara no linealidad producida esencialmente ante un número bajo de bits en la entrada.

Este resultado es bastante lógico, pues como ya ocurría en GPG, `OpenSSL` al cifrar con `enc`, añade una serie de cabeceras con información adicional (salt, padding...), lo que significa que si el mensaje original es muy pequeño, el overhead puede ser mucho mayor que el propio mensaje, haciendo que el caudal efectivo sea muy inferior a la media. Por otro lado, se puede llegar a la conclusión de que los algoritmos de cifrado antiguos (DES,3DES) son menos eficientes que los que se consideran más modernos (AES); además, cuanto mayor es el tamaño de la clave menor es el caudal efectivo.

Los comandos que reflejan los datos han sido ejecutados en equipos distintos a los del laboratorio (en los que se probaron el resto de algoritmos de cifrado).

¿Podría el ordenador en el que ha ejecutado la prueba generar un mensaje cifrado de 2MB antes en menos de un segundo?

Si se acude a las gráficas generadas se puede observar como es posible un thoughput de 2MB/s para cualquiera de los diferentes algoritmos de cifrado, además para esa cantidad de bytes ya no aparece la no linealidad, garantizándolo al completo.

El cálculo del caudal se podría hacer de manera manual, por ejemplo, engordando un fichero (de tamaño controlado) y midiendo el tiempo mediante el comando `time`.

```
(dit@lt205)-[~/cifrado]
$ echo `cat alice.txt` >> alice.txt
```

Como el anterior comando no engordaba lo suficiente el tamaño del fichero `alice.txt` se optó por la programación de un bucle que hiciera esa misma operación un total de 5 veces, empleando un fichero intermedio denominado `alice.orig`.

```
(dit@lt205)-[~/cifrado]
$ cp alice.txt alice.orig && for i in {1..5};
do cat alice.orig >> alice.txt; done
```

La salida del comando `time` te proporciona 3 contadores, el del sistema (`sys`), el de usuario (`user`) y el `real`. El primero de ellos indica el tiempo total de reloj, es decir, cuánto tiempo pasó desde que se ejecutó el programa hasta que terminó. El segundo hace referencia al tiempo total que la CPU pasó ejecutando código de usuario (el programa propiamente dicho). Y por último, el tiempo de sistema que es el tiempo total que la CPU pasó ejecutando código del kernel (llamadas al sistema, E/S, etc.). Por todo esto, si queremos medir la eficiencia del algoritmo puro, se tendrá en cuenta exclusivamente el tiempo de usuario.

```
$ time openssl enc -aes-256-cbc -salt -in alice.txt -out C.enc
-pass pass:MISECRETO
```

Tras la ejecución repetida del comando, se realizó una tabla con los valores obtenidos. Si bien el enunciado de la práctica propone medir valores hasta los 10MB, se ha optado por seguir engordando el archivo `alice.txt` para obtener una medida del caudal más ajustada a la realidad.

Tamaño 'alice.txt' (kB)	Tiempo medido (s)
891,264	0,01
5347,584	0,02
15053,848	0,06
32085,504	0,06
192513,024	0,48
1155078,144	2,62

¿Le sale una velocidad de codificación (throughput) coherente con el test del comando speed?

Si dividimos el tamaño del archivo de texto entre el tiempo de usuario y realizamos la media aritmética de los resultados, obtenemos que el throughput (kB/s) equivale a 330683,3, un valor muy alejado de la media expresada en la Figura 3. Esto se debe principalmente a que los comandos fueron ejecutados en distintos dispositivos con diferentes componentes, lo que corrompe la prueba. Por ello, si se procede a calcular el caudal medio en el mismo equipo donde se midió el tiempo, se obtiene lo siguiente:

```
(kali@kali)-[~/ciberseguridad]
$ openssl speed aes-256-cbc
Doing aes-256-cbc ops for 3s on 16 size blocks:
77569217 aes-256-cbc ops in 3.03s
Doing aes-256-cbc ops for 3s on 64 size blocks:
21725880 aes-256-cbc ops in 3.03s
Doing aes-256-cbc ops for 3s on 256 size blocks:
5715771 aes-256-cbc ops in 3.03s
Doing aes-256-cbc ops for 3s on 1024 size blocks:
1439956 aes-256-cbc ops in 3.03s
Doing aes-256-cbc ops for 3s on 8192 size blocks:
180805 aes-256-cbc ops in 3.02s
Doing aes-256-cbc ops for 3s on 16384 size blocks:
90320 aes-256-cbc ops in 3.03s
...
type              16 bytes     64 bytes    256 bytes
                  1024 bytes   8192 bytes  16384 bytes
aes-256-cbc       409606.43k   458896.48k   482916.63k
                  486638.60k   490448.53k   488383.79k
```

Si realizamos la media aritmética de los datos que aparecen en la parte inferior de la salida del comando, obtenemos que el throughput es de 469481,7433 kB/s, un valor mucho más cercano al calculado previamente.

3. Ejercicio simple de cifrado asimétrico

3.1. Generación de claves pública y privada

Mediante el siguiente comando, `OpenSSL` genera una clave privada RSA de 256 bits (por defecto) y la muestra por la salida estándar.

```
(dit@lt205)-[~/cifrado]
$ openssl genrsa
```

```
-----BEGIN PRIVATE KEY-----
MIIEvAIBADANBgkqhkiG9w0BAQEFAASCBKYwggSiAgEAAoIBAQCMDShYhbOi+a6i
qqNQGz1zxa2YRrff06d258paSy5L7Ei7xx44Be6W775UIAtUJElH5PBeteAyghnv
V/O6oqSumHRfDVJUS5/o7SBQcB0vbC6obtdLVwTW07yZ3gT+jO7BnhuX8xEirv4R
ua7G5S4p3eZGLOYpZeYHy0PmCuJP3JKemOeb4inif/YotTNHMgpcV+c+NDpoS11k
hzE+OGfisTTRO/v710/0QuS3xX9u6G3i7vnkE0Ek8UpzGOquhAFTMoaYHBR3p8Eo
8+GbTTJfyf8uM6F6iBIRUJJa29HsWIFL4BnuBx0W/PmqezEGi3KIR2ywemivvL8c
gdJWMrQzAgMBAAECggEAA46X/juTFuoRgeckk7XFZ0nXgV0WcXReNfpoHMf5Coub
Q02o2lJoP2QiaN+Rehcb11wcsMMcyRudzgLlWVGlLhwIJJvhgeuTyaYwImreiuRM
ARIjnLG+mTgye5P0e4SX0coSuFa/kipwoIGfS0Tp2b1dil28kMOaRotmk5KmMdIG
NuWA5LtrVF4jCIpEC2s+DKI9bubPSmZ7braWMMAUu4i2TNsHjIQhv5e0X6eYrkrB
hr3lvLPVTaTeC4lWgZ77jN4WMvru/H+C1fy0rGBbgS6V9MyUMMBNo/jBSKnMAnqB
qwHrTEqRbodO1ggd9+5HBdTn851kn0MsQTi4FEBa+QKBgQC/Kc/j3esp0mLd/dqn
N2DfcD2lY4xSlCbe4hxI1kkzPctHrXApeecsOAAnEeoLB87G5L/B2GfDeysxG4Sq
FZxicY2IfghtQDtEUjQotovyZJEwo/0birdca2CQIBfmuzCA8m/LTNtIaY396mlS
32joTtJpKb/Hs3wtYnmHrNudKwKBgQC7jW/EHArUuWAqmJ7vxcuPEKmWpywk2t4o
Czg+3dtx6myaifIOZ9dWTRIRfZD2w+2d15em/zOGqfpwVKzG3vZb7LJwmHrbrDw4
Uj5tv+pBg2XZUtbwKWoQlBzhBXYRwOE7GUps5VM/A+i8o9rOsElhrm7RdhDMGjKl
0FZgMrGRGQKBgFWIFL9EzPKCcsAW1B1bVtnCc6Krft538UhqbPNeBZSKwoiInpXA
OGH9kHvqtQhIbwlo0i+ebxb05idrw5kCYF1fp0757TVMtvfju/i3bLiArMHRre8D
yvTsCmGSKn+PI62eS8zvz3F/4QtKNePhmCPRGK37AxcMRLRdyjidbAnPAoGAL+dB
K8LJ1Wl/sYGH3tATXPgxiKQsN0GbyniLJit+WEs68AHZzGojawyGV+dNdmm2sEEX
n/TA0V1q9PP7qVPbcsEzFmUrxB3qx6vh/CAvkgogonBxK6g4SgG6mpLNhF1Sacjc
5JFyQL8b28qyIWaPZ56o+WEBi1AJraZX2Xw5NEkCgYA5JMtFixkCXfSyG+4PT4In
yfo3U2kiTY/POEOE33C047fxFTQ/ajas4LG9H+79+TjYqVw7M9MknAyP8BuD25OD
ZiBOgxcAzLSDlOwBTSjObrXD+fzQs6aETcNmpRFY/IQW0hpFO+uU0eilfdCdRpn4
tF8dLb7p0fdFVdnwLRl6aQ==
-----END PRIVATE KEY-----
```


Se puede especificar el fichero en el que se quiere almacenar (mediante la bandera `-out`), además del tamaño de clave, en este caso, se ha generado una clave privada RSA de 4096 bits y se ha almacenado en el fichero `rsa_private_key`.

```
(dit@lt205)-[~/cifrado]
$ openssl genrsa -out rsa_private_key 4096
```

Como se trata de un cifrado asimétrico, además de la clave privada será necesario la pública, la cual se puede generar de la anterior mediante la ejecución del siguiente comando.

```
$ openssl rsa -pubout -in rsa_private_key -out rsa_public_key
```

Si se muestra de manera legible el contenido de ambos ficheros, se pueden identificar los diferentes parámetros con los que se construye la clave RSA. En el `rsa_public_key` aparecen los parámetros públicos: el módulo n y el exponente público e.

```
(dit@klt205)-[~/cifrado]
$ openssl rsa -in rsa_public_key -pubin -text -noout
Public-Key: (4096 bit)
Modulus:
```

```
    00:d1:d9:dd:0b:a4:cd:d5:80:03:f8:29:ce:52:d5:
    ...
Exponent: 65537 (0x10001)
```

Por otro lado, en el `rsa_private_key` se incluyen los parámetros privados: los números primos p y q, el exponente privado d, y los valores derivados para optimizar el cálculo mediante el teorema asociado (d_P, d_Q, q_{Inv}).

```
(dit@klt205)-[~/cifrado]
$ openssl rsa -in rsa_private_key -text -noout
Private-Key: (4096 bit, 2 primes)
modulus:
    00:d1:d9:dd:0b:a4:cd:d5:80:03:f8:29:ce:52:d5:
    ...
publicExponent: 65537 (0x10001)
privateExponent:
    34:a0:e2:e2:ec:ae:02:81:5c:7d:7c:5e:96:3f:a3:
    ...
prime1:
    00:fd:ee:27:b0:9b:e1:df:5f:81:a7:21:f8:76:a3:
    ...
prime2:
    00:d3:8f:bb:d1:a3:91:b8:7f:30:9e:c0:13:8c:35:
    ...
exponent1:
    4e:52:44:9f:ae:94:ee:86:d0:c0:b2:33:a6:b4:26:
    ...
exponent2:
    10:ee:76:ec:9f:37:a1:59:64:8d:bf:f7:90:88:0f:
    ...
coefficient:
    74:35:e0:e3:61:47:79:8d:95:68:aa:ff:cc:26:26:
    ...
```

3.2. Cifrado y descifrado con un compañero

En este apartado se simulará una comunicación mediante cifrado asimétrico (en ambas direcciones). Primero se procederá a encriptar el fichero `Marga.txt` con la clave pública del destino. Para versiones actualizadas de `OpenSSL` la opción `rsautl` se encuentra obsoleta, por lo que en su defecto se empleará `pkeyutl`.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -pubin -inkey rsa_pub_compi
-encrypt -in M_tuyo.txt -out M_tuyo.rsa
```

A continuación se desencriptará el mensaje enviado por la compañera. Para probar el funcionamiento del cifrado asimétrico, primero se procederá a descifrar el fichero `M_compi.rsa` mediante la clave pública. Como es lógico ocurrirá un error, pues lo que se debe proporcionar es la clave privada.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -pubin -inkey rsa_public_key
-decrypt -in M_compi.rsa -out M_compi.txt
A private key is needed for this operation
pkeyutl: Error loading key
```

Mediante el siguiente comando queda demostrado la necesidad de la clave privada para descifrar aquellos mensajes encriptados bajo nuestra clave pública. Por último, mediante el comando `cat` se muestra el contenido del fichero recibido.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -inkey rsa_private_key -decrypt
-in M_compi.rsa -out M_compi.txt
```

```
(dit@lt205)-[~/cifrado]
$ cat M_compi.txt
Esto es un mensaje cifrado
```

4. Ejercicio simple de firma electrónica

En la ciberseguridad no solo basta con la confidencialidad, sino que es necesario un procedimiento que otorgue autenticidad a los mensajes. La firma digital viene a suplir esta ausencia. A efectos prácticos consiste en encriptar mediante tu clave privada, así el resto de individuos a través de tu clave pública podrá verificar que el mensaje ha sido generado por ti. La ejecución del siguiente comando, empleando la bandera `-sign`, permite lo anteriormente explicado.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -sign -inkey rsa_private_key -in M.txt
-out firma.bin
```

Si se quisiera verificar la firma digital de un fichero, se debe ejecutar el comando a continuación (empleando la bandera `-verifyrecover`).

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -verifyrecover -pubin -inkey rsa_public_key
-in firma.bin -out M_recuperado.txt
```

```
(dit@lt205)-[~/cifrado]
$ cat M_recuperado.txt
Este es un mensaje en claro
```

4.1. Generación de claves públicas y privadas en gpg

GPG tiene un almacén de claves (base de datos local en `/.gnupg/trustdb.gpg`) por cada usuario con las claves de los usuarios en los que se confía. Mediante el siguiente comando, se generará un par de claves pública y privada y se almacenará en el llavero (bbdd local) mediante un identificador. También se deberá establecer una contraseña para el acceso a este.

```
(dit@lt205)-[~/cifrado]
$ gpg --gen-key
gpg (GnuPG) 2.4.7; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Nota: Usa "gpg --full-generate-key" para el dialogo completo
de generacion de clave.

GnuPG debe construir un ID de usuario para identificar su clave.

Nombre y apellidos: Juan Antonio
Direccion de correo electronico: juarobmen@alum.us.es
Esta usando el juego de caracteres 'utf-8'.
Ha seleccionado este ID de usuario:
    "Juan Antonio <juarobmen@alum.us.es>"

Cambia (N)ombre, (D)ireccion o (V)ale/(S)alir? V
```

Se puede comprobar su correcta generación listando las claves del llavero.

```
(dit@lt205)-[~/cifrado]
$ gpg --list-keys
/home/kali/.gnupg/pubring.kbx
------------------------------
pub   ed25519 2025-10-31 [SC] [expires: 2028-10-30]
      F3496EA0DCBD2E38E1CDB7693F8D76956DDF1AD4
uid           [ultimate] Juan Antonio <juarobmen@alum.us.es>
sub   cv25519 2025-10-31 [E] [expires: 2028-10-30]
```

A continuación se exporta la clave pública. Es imprescindible especificar en el comando su identificador, en este caso el correo asociado `juarobmen@alum.us.es`.

```
(dit@lt205)-[~/cifrado]
$ gpg --armor --output juanantonio.gpg --export juarobmen@alum.us.es
```

Por otro lado, importaremos la clave pública del compañero, la cual se añadirá al llavero del equipo.

```
(dit@lt205)-[~/cifrado]
$ gpg --import compi.gpg
gpg: key D11DE109407F5813: public key "Compi <compi@us.es>" imported
gpg: Total number processed: 1
```

```
gpg:               imported: 1
```

De nuevo, si mostramos un listado de nuestro llavero deberá aparecer. Cabe destacar que la clave importada viene marcada en el `uid` como `unknown`, y es que, la confianza en la clave de un individuo se puede clasificar en 5 niveles:

- `Unknown`: Nada se sabe sobre el juicio del propietario en la firma de claves.
- `None`: Se sabe que el propietario firma incorrectamente otras claves.
- `Marginal`: El propietario entiende las implicaciones de firmar claves y valida correctamente las claves antes de firmarlas.
- `Full`: El propietario tiene un excelente conocimiento sobre la firma de claves, y su firma en una clave sería tan válida como la suya propia.
- `Ultimate`: Este nivel se reserva para su propia clave privada; implica confianza total y definitiva, ya que usted es el propietario de la clave y no necesita validación externa.

```
(dit@lt205)-[~/cifrado]
$ gpg --list-keys
/home/kali/.gnupg/pubring.kbx
------------------------------
pub   ed25519 2025-10-31 [SC] [expires: 2028-10-30]
      F3496EA0DCBD2E38E1CDB7693F8D76956DDF1AD4
uid           [ultimate] Juan Antonio <juarobmen@alum.us.es>
sub   cv25519 2025-10-31 [E] [expires: 2028-10-30]

pub   ed25519 2025-10-31 [SC] [expires: 2028-10-30]
      553175E406D44E99670DB437D11DE109407F5813
uid           [ unknown] Compi <compi@us.es>
sub   cv25519 2025-10-31 [E] [expires: 2028-10-30]
```

Se procederá a validar la clave importada marcándola como de confianza, en este caso se ha optado por asignarle `marginal`.

```
(dit@lt205)-[~/cifrado]
$ gpg --edit-key compi@us.es
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.


pub  ed25519/D11DE109407F5813
     created: 2025-10-31  expires: 2028-10-30  usage: SC
     trust: unknown       validity: unknown
sub  cv25519/72511A32734A1514
     created: 2025-10-31  expires: 2028-10-30  usage: E
[ unknown] (1). Compi <compi@us.es>
```

```
gpg> trust
pub  ed25519/D11DE109407F5813
     created: 2025-10-31  expires: 2028-10-30  usage: SC
     trust: unknown       validity: unknown
sub  cv25519/72511A32734A1514
     created: 2025-10-31  expires: 2028-10-30  usage: E
[ unknown] (1). Compi <compi@us.es>

Please decide how far you trust this user to correctly
verify other users' keys(by looking at passports, checking
fingerprints from different sources, etc.)

  1 = I don't know or won't say
  2 = I do NOT trust
  3 = I trust marginally
  4 = I trust fully
  5 = I trust ultimately
  m = back to the main menu

Your decision? 3

pub  ed25519/D11DE109407F5813
     created: 2025-10-31  expires: 2028-10-30  usage: SC
     trust: marginal      validity: unknown
sub  cv25519/72511A32734A1514
     created: 2025-10-31  expires: 2028-10-30  usage: E
[ unknown] (1). Compi <compi@us.es>
Please note that the shown key validity is not necessarily correct
unless you restart the program.

gpg> save
Key not changed so no update needed.
```

A través del siguiente comando generamos la firma digital de un documento pasado como parámetro (el documento a firmar `alice.txt` es la entrada y la firma `alice.sig` es la salida). Este paso lo realizará el compañero.

```
(dit@lt205)-[~/cifrado]
$ gpg --armor --output alice.sig --sign alice.txt
```

Por último, simplemente se descifar (autenticación y recuperación del documento original). Se puede apreciar como GPG reconoce la firma con su identificador del llavero. El aviso quiere decir que la firma es correcta pero no tienes una cadena de confianza completa para afirmar que el propietario de la clave es realmente tu compañero.

```
(dit@lt205)-[~/cifrado]
$ gpg --output alice.txt --decrypt alice.sig
gpg: Signature made Sat 01 Nov 2025 12:53:20 PM EDT
gpg:                using EDDSA key
553175E406D44E99670DB437D11DE109407F5813
gpg: Good signature from "Compi <compi@us.es>" [unknown]
```

```
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature
              belongs to the owner.
Primary key fingerprint: 5531 75E4 06D4 4E99 670D
                         B437 D11D E109 407F 5813
```

5. Ejercicio final

El escenario consta de 2 individuos A y B. El usuario A cifra un mensaje (M) usando cifrado simétrico (p.ej. AES128). Para ello usa la frase de paso (PF) que equivaldrá a la clave de sesión. El usuario A envía el mensaje cifrado al usuario B (sin comunicarle la clave de paso). A continuación, ambos usuarios A y B intercambian sus claves públicas. Después, el usuario A cifra la frase de paso (FP) con la clave pública de B. (se supone la salida S=B+(FP)). A continuación A firma (p.ej. cifra con su clave privada) S, produciendo como salida X=A-(B+(FP)). Envía dicho mensaje X al usuario B, el cual debe ser capaz de:

1. Verificar que el mensaje con la clave de sesión viene de A (autenticación).
2. Averiguar cuál es la clave de sesión (frase de paso o FP).
3. Descifrar el mensaje original M.

En A primero se ha creado un fichero que contiene el texto en claro (`M.txt`)

```
(dit@lt205)-[~/cifrado]
$ echo 'Si lees esto es porque lo has conseguido descifrar' > M.txt
```

A continuación se ha cifrado dicho mensaje con un algoritmo de clave simétrica (AES128). Se ha empleado para ello GPG. Al encriptarlo solicita una passphrase (PF), que será en este caso `juarobmen`. El resultado se ha almacenado en el fichero `C.AES`.

```
(dit@lt205)-[~/cifrado]
$ gpg -o C.AES -c --cipher-algo AES128 M.txt
```

Posteriormente, se ha generado el par de claves públicas y privadas tanto en `A` como en `B`. Como el ejercicio recomienda usar un programa distinto al empleado para la clave simétrica, se ha optado por `OpenSSL`. Debido al funcionamiento de este, primero se han creado las claves privadas `rsa_private_key` de 4096 bits de longitud. Luego se han calculado las públicas (en el caso de `A` `rsa_public_key1` y en el caso de `B` `rsa_private_key2`) y se han intercambiado.

```
(dit@lt205)-[~/cifrado]
$ openssl genrsa -out rsa_private_key 4096
```

```
(dit@lt205)-[~/cifrado]
$ openssl rsa -pubout -in rsa_private_key -out rsa_public_key1
writing RSA key
```

```
(dit@lt205)-[~/cifrado]
$ openssl rsa -pubout -in rsa_private_key -out rsa_public_key2
writing RSA key
```

Tal y como indica el enunciado, se ha creado en `A` un fichero denominado `passphrase` que contiene en texto claro la PF para desencriptar el `M.txt`.

```
(dit@lt205)-[~/cifrado]
$ echo 'juarobmen' > passphrase
```

Ese archivo antes mencionado será crifado mediante la clave pública de `B` (`rsa_public_key2`). El resultado se ha almacenado en `passphrase.rsa`. Esto garantiza confidencialidad.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -pubin -inkey rsa_public_key2 -encrypt
-in passphrase -out passphrase.rsa
```

`A` firma con su clave privada un hash (MD5) del fichero `passphrase.rsa` para otorgar autenticidad e integridad. Se genera un hash porque al aplicar el siguiente comando, la longitud de los datos de entrada no en formato `raw` es de 512B, lo cual es demasiado grande, puesto que solo admite un máximo de 64B. Durante la realización del ejercicio se ha intentado buscar opciones que consiguiesen firmar digitalmente y cifrar un archivo de manera simultánea, pero aparentemente `OpenSSL` a diferencia de GPG no ofrece dichas posibilidades.

```
(dit@lt205)-[~/cifrado]
$ md5sum passphrase.rsa | openssl pkeyutl -sign
-inkey rsa_private_key > sign.bin
```

Como resumen, a `B` se le ha proporcionado tres ficheros: `C.AES`, `rsa_public_key1` y `sign.bin`.

Verificar que el mensaje con la clave de sesión viene de A (autenticación)

Para lograr esto, solo se debe comprobar si `sign.bin` al aplicarle la clave pública de `A` (`rsa_public_key1`) el valor del hash resultante se ajusta al del `passphrase.rsa`.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -verifyrecover -pubin -inkey rsa_public_key1
-in sign.bin -out recover.txt
```

```
(dit@lt205)-[~/cifrado]
$ cat recover.txt
1e1eb01214c1a8e4a3a9a1d46e6ea6ba  passphrase.rsa
```

Averiguar cuál es la clave de sesión (frase de paso o FP)

Una vez `B` se ha asegurado de que el fichero `passphrase.rsa` no ha sido modificado y ha sido generado por `A`, puede a través de su clave privada (`rsa_private_key`) descifrar el contenido del fichero que contiene la PF. El resultado se ha vertido en un fichero denominado `passphrase`.

```
(dit@lt205)-[~/cifrado]
$ openssl pkeyutl -inkey rsa_private_key -decrypt
-in passphrase.rsa -out passphrase
```

```
(dit@lt205)-[~/cifrado]
$ cat passphrase
juarobmen
```

Descifrar el mensaje original M

Con el conocimiento de la PF, tan solo se debe mediante GPG desencriptar el `C.AES` mediante este mismo. El resultado de la operación se ha almacenado en un fichero denominado `M.txt`.

```
(dit@lt205)-[~/cifrado]
$ gpg -o M.txt -d C.AES
gpg: AES.CFB encrypted data
gpg: encrypted with 1 passphrase
```

```
(dit@lt205)-[~/cifrado]
$ cat M.txt
Si lees esto es porque lo has conseguido descifrar
```